PAR Laboratory Assignment

Lab 4: Parallel Task Decomposition
Implementation and Analysis

Oriol Valencia

Universitat Politècnica de Catalunya
Departament d'Arquitectura de Computadors

Spring 2025–2026

Contents

1 Iterative Task Decomposition **2**
1.1 Iterative: Tile Task Decomposition . . . 2
1.1.1 Implementation . . . 2
1.1.2 Modelfactor Analysis . . . 2
1.1.3 Paraver Analysis . . . 2
1.1.4 Strong Scalability . . . 2
1.2 Iterative: Fine Grain Task Decomposition . . . 3
1.2.1 Implementation . . . 3
1.2.2 Modelfactor Analysis . . . 3
1.2.3 Paraver Analysis . . . 4
1.2.4 Strong Scalability . . . 5

2 Recursive Task Decomposition **6**
2.1 Recursive: Leaf Task Decomposition . . . 6
2.1.1 Implementation . . . 6
2.1.2 Modelfactor Analysis . . . 6
2.1.3 Paraver Analysis . . . 7
2.1.4 Strong Scalability . . . 7
2.2 Recursive: Improved Leaf Task Decomposition . . . 8
2.2.1 Implementation . . . 8
2.2.2 Modelfactor Analysis . . . 8
2.2.3 Paraver Analysis . . . 9
2.2.4 Strong Scalability . . . 9
2.3 Recursive: Tree Task Decomposition . . . 11
2.3.1 Implementation . . . 11
2.3.2 Modelfactor Analysis . . . 11
2.3.3 Paraver Analysis . . . 11
2.3.4 Strong Scalability . . . 11

A Final Comparison Between All Parallel Strategies **12**
A.1 Summary of Elapsed Execution Times . . . 12
A.2 Best Implementation Discussion . . . 12

Chapter 1

Iterative Task Decomposition

1.1 Iterative: Tile Task Decomposition

1.1.1 Implementation

The **original tile iterative** strategy maps the $ROWS \times COLS$ image onto a grid of square tiles of size $TILE \times TILE$ pixels. The outer double loop iterates over tile origins with stride `TILE`; each iteration body is wrapped in a single `#pragma omp task`, so all tiles are eligible for parallel execution. The task takes `x` and `y` as `firstprivate` to avoid the data-race that would arise if the loop variables were shared across concurrently executing tasks.

The parallel region is opened with `#pragma omp parallel` followed immediately by `#pragma omp single`, so only one thread generates the tasks while all others are available to execute them.

The key parallel section (double loop with `#pragma omp task` over tile origins) is implemented in `mandel-omp-iter.c`.

Dependencies and data-sharing between tasks are avoided by construction: each task writes only to the memory region $[y, y + TILE) \times [x, x + TILE)$, which is disjoint from every other task's region. Histogram updates use `#pragma omp atomic` to protect the shared counter array.

1.1.2 Modelfactor Analysis

Insert Modelfactors table for `mandel-omp-iter` (16 threads) and identify the main bottlenecks (e.g. scheduling overhead, load imbalance, synchronisation).

1.1.3 Paraver Analysis

Insert Paraver screenshots (16 threads) and comment on the task creation pattern, load distribution, and any visible idle periods.

1.1.4 Strong Scalability

Insert the strong scalability (speedup vs. number of threads) plot for `mandel-omp-iter` and explain the observed trend.

1.2 Iterative: Fine Grain Task Decomposition

1.2.1 Implementation

The **fine grain iterative** strategy extends the Tile version by exploiting the additional parallelism that exists *inside* each tile. Rather than completing the border check and fill/compute sequentially within each tile task, the fine grain version exposes three independent sub-tasks per tile:

1. A task that computes the *vertical* borders of the tile (left and right columns), setting `eV`.
2. A task that computes the *horizontal* borders (top and bottom rows, including all four corners), setting `eH`.
3. A task (spawned after `#pragma omp taskgroup` ensures both border tasks have finished) that either fills the tile uniformly or, in the non-uniform case, spawns one child task per row to compute all interior pixels.

The outer tile task from the Tile version is **kept** as a wrapper; the three inner tasks are created inside it.

The key parallel section (three inner tasks per tile: vertical borders, horizontal borders, and fill/compute) is implemented in `codis/fine-grain.c`.

The `#pragma omp taskgroup` enforces that the result tasks begin only after *both* border tasks complete, because the fill/compute decision depends on `eV` and `eH`. Histogram increments remain protected by `#pragma omp atomic`. Display updates use `#pragma omp critical` to serialize X11 calls.

Comparison with Iterative Tile. The Tile version creates $\left(ROWS/TILE\right)^2$ tasks in total. The Fine Grain version creates up to $3 + 2 \cdot TILE$ tasks *per tile*, increasing the task count by roughly $2 \cdot TILE$ fold. This exposes more parallelism at the cost of higher task-creation and synchronisation overhead; whether the trade-off is beneficial depends on tile size and thread count.

1.2.2 Modelfactor Analysis

Tables 1.1–1.3 summarise the Modelfactors output for `mandel-omp-fine-grain` obtained on `par2317`.

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.49	1.75	0.88	0.45	0.30	0.24	0.19	0.17	0.15	0.13
Speedup	1.00	1.99	3.96	7.77	11.46	14.74	18.14	21.08	23.48	26.28
Efficiency	1.00	0.99	0.99	0.97	0.95	0.92	0.91	0.88	0.84	0.82

Table 1.1: Fine Grain – whole program execution metrics. Analysis: par2317, 2026-05-15.

Overview of the Efficiency metrics in parallel fraction, ϕ=99.99%										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.94%	99.41%	98.93%	97.15%	95.52%	92.18%	90.79%	87.96%	83.98%	82.32%
Parallelization strategy efficiency	99.94%	99.47%	98.99%	97.39%	95.91%	93.08%	91.73%	88.79%	85.77%	83.68%
Load balancing	100.00%	99.97%	99.77%	98.96%	98.61%	96.52%	94.72%	92.67%	89.14%	87.67%
In execution efficiency	99.94%	99.51%	99.22%	98.42%	97.26%	96.43%	96.84%	95.82%	96.23%	95.45%
Scalability for computation tasks	100.00%	99.93%	99.94%	99.75%	99.58%	99.04%	98.97%	99.06%	97.91%	98.37%
IPC scalability	100.00%	99.90%	99.88%	99.87%	99.87%	99.79%	99.77%	99.84%	99.76%	99.84%
Instruction scalability	100.00%	100.10%	100.10%	100.10%	100.09%	100.10%	100.09%	100.09%	100.10%	100.09%
Frequency scalability	100.00%	99.94%	99.96%	99.79%	99.62%	99.15%	99.11%	99.13%	98.05%	98.43%

Table 1.2: Fine Grain – efficiency breakdown (ϕ=99.99%). Analysis: par2317, 2026-05-15.

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0	8448.0
LB (number of explicit tasks executed)	1.0	0.95	0.64	0.45	0.31	0.38	0.38	0.4	0.22	0.31
LB (time executing explicit tasks)	1.0	1.0	1.0	0.99	0.99	0.97	0.96	0.96	0.93	0.92
Time per explicit task (average us)	411.87	413.36	413.91	416.69	419.9	426.35	428.96	434.22	447.72	449.23
Overhead per explicit task (synch %)	0.0	0.33	0.6	1.7	2.42	4.36	4.94	6.57	8.01	9.28
Overhead per explicit task (sched %)	0.06	0.19	0.35	0.85	1.52	2.57	3.24	4.67	6.53	7.59
Number of taskwait/taskgroup (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0

Table 1.3: Fine Grain – explicit task statistics. Analysis: par2317, 2026-05-15.

Observations. The fine grain version scales remarkably well across all thread counts. At 32 threads the elapsed time drops from 3.49 s to 0.13 s, yielding a speedup of 26.28× and a global efficiency of 82 % (Table 1.1).

The efficiency breakdown (Table 1.2) shows that the dominant loss source is *load imbalance*: the load-balancing factor falls to 87.67 % at 32 threads while in-execution efficiency remains above 95 %. IPC and instruction scalability are virtually perfect (≥99.8 %), confirming that the computation itself is highly regular and portable across cores.

The task statistics (Table 1.3) reveal 8 448 explicit tasks in total, providing ample work to keep all threads busy. Synchronisation overhead grows from 0 % at 1 thread to 9.28 % at 32 threads, while scheduling overhead reaches 7.59 %. These costs are still modest relative to the useful computation per task (≈412 µs average), so the task granularity remains appropriate even at high parallelism. The slight decline in load balance at 32 threads arises because the 8 448 tasks do not divide evenly among threads and some tiles are computationally heavier (Mandelbrot border regions).

Compared with the Iterative Tile version, the fine grain decomposition exposes substantially more parallelism by splitting each tile's border and interior work into independent tasks. This allows threads to overlap border computation across different tiles and fills in potential idle gaps, leading to the near-linear speedup curve observed here.

1.2.3 Paraver Analysis

Insert Paraver screenshots (16 threads) for Fine Grain and compare with the Tile trace. Comment on whether the extra tasks reduce idle time or introduce additional overhead.

1.2.4 Strong Scalability

Insert the strong scalability plot for Fine Grain and compare with the Tile speedup curve. Discuss whether additional task granularity improves or hurts scalability at high thread counts.

Chapter 2

Recursive Task Decomposition

2.1 Recursive: Leaf Task Decomposition

2.1.1 Implementation

The **leaf recursive** strategy implements the Mandelbrot computation through a divide-and-conquer quadtree recursion. Starting from the full $ROWS \times COLS$ matrix, each call to `mandel_tiled_rec` inspects the four borders of its sub-tile:

- If all border pixels share the same iteration count (`equal = 1`), the entire sub-tile is filled with that value via a **leaf task**.
- If the sub-tile is at or below the minimum tile size (`NCols <= TILE`), all interior pixels are computed individually via a **leaf task**.
- Otherwise the sub-tile is split into four quadrants (or two halves when only columns remain to be halved) and `mandel_tiled_rec` is called recursively for each quadrant – *sequentially*, with no task at the recursive call sites.

Tasks are therefore created only at the *leaves* of the recursion tree.

The complete implementation (leaf tasks for fill and compute, sequential recursion at internal nodes) is in `codis/mandel-omp-rec(1).c`.

Because the recursive calls are sequential, only one thread walks the tree and generates leaf tasks; all other threads execute those tasks. Data races are avoided: each leaf task writes an exclusive region of `M`. Shared histogram updates use `#pragma omp atomic`.

Limitation. With the leaf strategy, task creation is inherently *serial* – only the thread executing the single region can recurse and spawn. This means that at the start of execution, while the tree is being traversed, most threads remain idle waiting for tasks. The Improved Leaf version (Section 2.2) addresses this bottleneck.

2.1.2 Modelfactor Analysis

Insert Modelfactors table for Leaf Recursive (16 threads). Identify the main issues: expected large fraction of time in scheduling/synchronisation due to serial task generation.

2.1.3 Paraver Analysis

Insert Paraver screenshots (16 threads) for Leaf Recursive and comment on the serial task-generation phase, the degree of parallelism achieved during leaf execution, and load balance.

2.1.4 Strong Scalability

Insert the strong scalability plot for Leaf Recursive and discuss the observed speedup. Is it limited to less than 2× for more than 2 threads? Why?

2.2 Recursive: Improved Leaf Task Decomposition

2.2.1 Implementation

The **improved leaf** version addresses the main bottleneck of the plain leaf strategy: serial task generation. In the leaf version, the entire recursion is driven by a single thread, so threads sit idle until enough leaf tasks have accumulated. The improvement introduces a *depth cutoff* parameter `d` (initialised to `0` at the top-level call) and a compile-time constant `CUTOFF`.

- When the current recursion depth satisfies `d` $<$ `CUTOFF`, each recursive quadrant call is wrapped in a `#pragma omp task`, allowing multiple threads to explore different branches of the recursion tree *concurrently*.
- When `d` $\geq$ `CUTOFF`, the recursive calls are sequential again, cutting off task proliferation at deep levels where the overhead would dominate.
- Leaf tasks (fill or compute) are only spawned when `d` $\leq$ `CUTOFF`; beyond the cutoff they run inline to avoid the overhead of tiny tasks.

The value `CUTOFF = 16` was chosen empirically so that the number of tasks at the cutoff level matches or exceeds the number of available threads.

The complete implementation (depth-cutoff mechanism with `CUTOFF = 16`) is in `codis/leaf-improved.c`.

The `#pragma omp taskgroup` ensures that all four quadrant tasks (and their descendants) finish before control returns to the caller, preserving the border-read correctness that the leaf version relies on.

Improvements over plain Leaf.

- Multiple threads now explore the recursion tree in parallel down to depth `CUTOFF`, eliminating the serial task-generation bottleneck.
- Tasks beyond `CUTOFF` run sequentially, avoiding excessive fine-grained task overhead at the bottom of the tree.
- The result is speedups larger than 2× for more than 2 threads.

2.2.2 Modelfactor Analysis

Tables 2.1–2.3 summarise the Modelfactors output for `mandel-omp-rec-improved-leaf` obtained on `par2317`.

Observations. The improved leaf version exposes a clear performance ceiling. Elapsed time decreases from 1.82 s (1 thread) to 0.38 s at 8 threads (speedup 4.76×) and then remains *flat* all the way to 32 threads (Table 2.1). Efficiency collapses to 15 % at 32 threads.

The efficiency breakdown (Table 2.2) identifies *load imbalance* as the dominant bottleneck: the load-balancing factor falls from 100 % at 1 thread to only 17.31 % at 32 threads. This is a direct consequence of the small total task count: only **66 explicit tasks** are

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.82	0.99	0.58	0.38	0.38	0.38	0.38	0.38	0.38	0.38
Speedup	1.00	1.84	3.12	4.76	4.76	4.74	4.74	4.74	4.73	4.74
Efficiency	1.00	0.92	0.78	0.60	0.40	0.30	0.24	0.20	0.17	0.15

Table 2.1: Improved Leaf – whole program execution metrics. Analysis: par2317, 2026-05-15.

Overview of the Efficiency metrics in parallel fraction, ϕ=99.98%										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.96%	91.86%	77.89%	59.57%	39.63%	29.65%	23.69%	19.77%	16.91%	14.81%
Parallelization strategy efficiency	99.96%	92.02%	78.01%	59.70%	39.78%	29.83%	23.91%	20.00%	17.18%	15.11%
Load balancing	100.00%	92.07%	80.14%	66.58%	45.42%	31.87%	27.40%	22.91%	18.37%	17.31%
In execution efficiency	99.96%	99.94%	97.34%	89.66%	87.57%	93.59%	87.25%	87.31%	93.54%	87.31%
Scalability for computation tasks	100.00%	99.83%	99.85%	99.78%	99.63%	99.40%	99.10%	98.82%	98.42%	98.04%
IPC scalability	100.00%	99.94%	99.95%	99.97%	99.94%	99.92%	99.90%	99.93%	99.93%	99.91%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.89%	99.89%	99.82%	99.68%	99.48%	99.19%	98.89%	98.49%	98.12%

Table 2.2: Improved Leaf – efficiency breakdown (ϕ=99.98%). Analysis: par2317, 2026-05-15.

generated (Table 2.3), making it impossible to distribute work evenly across more than a handful of threads.

The synchronisation overhead per task tells the same story: it grows from 0.04 % at 1 thread to **379 %** at 32 threads, meaning threads spend nearly four times as long waiting at synchronisation points (`taskwait`/ `taskgroup`) as they do executing useful computation. This is caused by the 1 004 `taskwait`/`taskgroup` operations that must serialise access between recursion levels, each becoming a bottleneck when more threads contend for a small pool of tasks. Scheduling overhead, by contrast, is negligible ($\leq$0.02 %), confirming that the overhead is synchronisation-bound, not dispatch-bound.

The saturation at 8 threads matches the CUTOFF-induced task count: with `CUTOFF=16`, the recursion tree generates at most $4^{\lceil \log_4 66 \rceil}$ leaf tasks, and at 8 threads the work is fully distributed; additional threads find no tasks and remain idle.

Compared with the plain Leaf version, the improved leaf eliminates the serial task-generation phase (multiple threads now traverse the recursion tree in parallel up to the cutoff), but the fundamental limit of 66 tasks prevents further scaling beyond 8 threads. Increasing `CUTOFF` would generate more tasks and potentially improve scalability at the cost of higher synchronisation overhead per task.

2.2.3 Paraver Analysis

Insert Paraver screenshots (16 threads) for Improved Leaf. Include both the non-improved and the improved traces for direct comparison.

2.2.4 Strong Scalability

Insert the strong scalability plot for Improved Leaf and compare with plain Leaf. Confirm that speedups larger than 2× are achieved for more than 2 threads.

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	66.0	66.0	66.0	66.0	66.0	66.0	66.0	66.0	66.0	66.0
LB (number of explicit tasks executed)	1.0	0.87	0.69	0.49	0.55	0.59	0.55	0.55	0.47	0.55
LB (time executing explicit tasks)	1.0	0.93	0.79	0.69	0.55	0.47	0.4	0.3	0.24	0.27
Time per explicit task (average us)	27349.36	27384.63	27386.04	28433.09	34192.49	40782.88	41755.11	37641.56	37773.34	41457.39
Overhead per explicit task (synch %)	0.04	8.71	28.37	65.43	122.21	159.14	210.86	295.23	355.38	379.16
Overhead per explicit task (sched %)	0.0	0.0	0.01	0.01	0.02	0.01	0.02	0.01	0.02	0.02
Number of taskwait/taskgroup (total)	1004.0	1004.0	1004.0	1004.0	1004.0	1004.0	1004.0	1004.0	1004.0	1004.0

Table 2.3: Improved Leaf – explicit task statistics. Analysis: par2317, 2026-05-15.

2.3 Recursive: Tree Task Decomposition

2.3.1 Implementation

The **tree recursive** strategy removes the leaf restriction entirely: tasks are created at *every* level of the recursion tree, not only at the leaves. Every recursive quadrant call becomes a task, so the task graph mirrors the full quadtree structure of the computation.

In the implementation used here (`mandel-omp-rec(2).c`), the tree strategy shares the same depth-cutoff infrastructure as the Improved Leaf version. The crucial difference is that task creation at *internal* nodes (the recursive calls) is enabled throughout the upper portion of the tree ($d <$ `CUTOFF`), and leaf tasks are also created in this region ($d \leq$ `CUTOFF`). Below the cutoff depth, both recursive and leaf calls revert to inline execution to control total task count.

The complete implementation (task creation at both internal nodes and leaves, with the same depth-cutoff as the Improved Leaf version) is in `codis/mandel-omp-rec(2).c`.

The `#pragma omp taskgroup` at each internal node enforces that all four child tasks (and their entire subtrees) complete before the enclosing task terminates, preserving the sequential semantics of the recursion.

Comparison with Improved Leaf. Both versions use the same cutoff mechanism; the only semantic difference is that the Tree version exposes quadrant-level parallelism from the very first recursion level ($d = 0$), giving all threads work to do immediately. In contrast, the Improved Leaf version also does this (because it too uses tasks at internal nodes when $d <$ `CUTOFF`); the distinction lies in the definition of "leaf": in the Tree version, *every* node with children is an internal task node, whereas the Improved Leaf restricts task creation to a shallower region. In practice, with `CUTOFF = 16`, both versions create tasks up to the same depth.

2.3.2 Modelfactor Analysis

Insert Modelfactors table for Tree Recursive (16 threads) and compare with Improved Leaf. Does the extra task creation at internal nodes improve useful computation or increase overhead?

2.3.3 Paraver Analysis

Insert Paraver screenshots (16 threads) for Tree Recursive and compare with the Improved Leaf trace. Comment on task density, load balance, and any synchronisation gaps introduced by the `taskgroup` barriers.

2.3.4 Strong Scalability

Insert the strong scalability plot for Tree Recursive and compare with Improved Leaf. Identify which version achieves better scalability and explain why.

Appendix A

Final Comparison Between All Parallel Strategies

A.1 Summary of Elapsed Execution Times

Table A.1 presents the elapsed execution times (in seconds) for all five parallel versions, obtained from the output files generated by `submit-strong-omp.sh`.

Table A.1: Elapsed execution times (seconds) for each version and number of threads.

Version	**Number of threads**					
	1	**4**	**8**	**12**	**16**	**32**
Iterative: Tile (provided, completed)						
Iterative: Fine grain						
Recursive: Leaf (provided, completed)						
Recursive: Improved Leaf						
Recursive: Tree						
Best parallel implementation		*Version:*			*Reason why:*	

A.2 Best Implementation Discussion

Fill in Table A.1 and write the discussion once experiments are complete.